Week 3

17 February 2026 11:02

Overall Project Discussion and Review

Reviewed the great progress that Rhodri is making:
He is largely focusing on **optimising the several thickness parameters associated with a multi layered blanket.**

We are finding that the **heterogenous model has a much lower TBR values than the homogeneous.**

- **Eg. 1.4 ---> 1.05**
- Could be due to "neutron traps" in the water
- Or due to the steel separator material being introduced
- **Suggesting looking into heat maps of reactions to help diagnose this.**
    - We do know that leakage fractions are still low

- Spent time helping him with what direction to go down
    - suggesting to investigate what effect the coolant is having specifically
    - suggesting to investigate what effect the first layer is having
    - Also suggested to do a point by point comparison of TBR against the homogenous model blanket thickness sweep
        - Though for to narrow this down we are including a proportional mix of stell into the homogenous model too to have a fair comparison4

In starting to look at including Iron we found something profound

- Fe56 has a spallation cross section (~ 0.5 barn) once neutrons hit 11+ MeV.
    - (and radiative capture is @ 10^-3 barn, more likely to be a multiplier)
- This means that iron can act as a further multiplier
    - This also means that saturation is reached at a larger thickness
- **But as its very endothermic, it can be concerning for heating**

Git work Review

Partner Help:

- Partner had their branch severely out of date (30 pulls required)
- Could not pull onto the file as too much conflicts
- **Made a backup branch before I attempted to fix:**
    - **"git checkout -b duplicated-branch-name"**
    - This makes a duplicated the current branch you are in into a new branch.
- **To solve can use**
    - **"git config pull.rebase false"**
    - This will setup git to setup a merge request in this instance
- Then rhodri just accepted all incoming changes, and he was happy to import any of his local changes from the backup branch.

ML work:
Had to import new packages on my PC build, which was local to that machine and not portable
How can I make the ML version portable

- **Firstly duplicated the branch to have a separate "ml-sandbox"**
    - **"git checkout -b duplicated-branch-name"**
- **Then decided to simply update the requirements.txt**
    - As the setup is complex and I don't trust package managers for this
- Note that this does mean that the requirements.txt will just default to using the **CPU not the GPU SDKs**
    - **SDK:** the software that can interface with an external processor like a GPU
    - The reason for this was that my PC build is also using the CPU, as it is **unlikely for the ML to be the runtime bottleneck.**

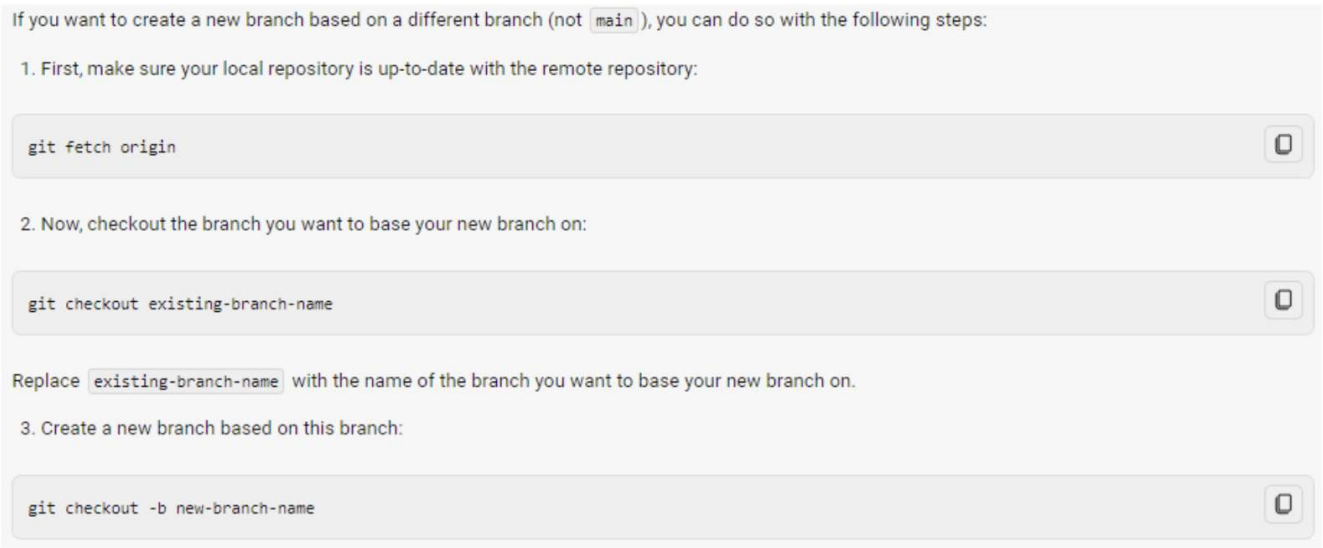
So I just added this to the requirements.txt

```
# --- Machine Learning ---
# Installing these will automatically pull the CPU version of torch
```

```
# because of the extra-index-url defined at the top.
botorch
ax-platform
```

Venv & Git Notes

To clone a branch



Exporting and Importing requirements.txt

This guide covers how to create (export) and use (import) a requirements.txt file to share or reproduce Python environments.

1. Exporting Dependencies (Creating the File)

There are two main approaches to creating a requirements.txt file.

Method A: The "Snapshot" (pip freeze)

This exports **every** single package currently installed in your environment, including dependencies of dependencies.

- **Best for:** Deploying to production servers where you need an exact replica of the environment.

Command:
pip freeze > requirements.txt

- **Result:** A long file with exact version numbers (e.g., pandas==2.0.3, numpy==1.24.3).

Method B: The "Top-Level" (Manual)

You manually list only the packages you strictly import in your code.

- **Best for:** Development, sharing libraries, or keeping things clean. It lets pip resolve the latest compatible sub-dependencies when installing later.
- **How to do it:**
  Run pip list to see what you have installed.
  Create a file named requirements.txt.
  Type only the names of the libraries you use (e.g., flask, requests, pandas).

2. Installing Dependencies (Importing)

To install the packages listed in a requirements.txt file into your current environment.

Command:

pip install -r requirements.txt

Common Flags:

- **Upgrade packages:** pip install --upgrade -r requirements.txt (Updates packages if newer versions exist).
- **User only:** pip install --user -r requirements.txt (Installs to your user directory if you don't have admin rights/virtualenv).

3. Best Practices & Syntax

Version Pinning

You can restrict which versions are installed to avoid breaking changes.

- package==1.0.0 : Installs exactly version 1.0.0.
- package>=1.0.0 : Installs version 1.0.0 or higher.
- package<=2.0.0 : Installs version 2.0.0 or lower.
- package : Installs the latest available version.

Comments

You can use # to add comments or categorize your packages.

Plaintext

```
# Data Science
pandas
numpy

# Web Framework
flask
gunicorn
```

Extra Index URLs

If you need packages from a custom source (not just PyPI), you can add the URL at the top of the file.

```
--extra-index-url https://download.pytorch.org/whl/cu118
torch
```

Virtual Environments

Always run these commands inside a virtual environment (.venv) to avoid messing up your global system Python installation.

```
# Check if you are in a venv (Linux/Mac)
which python
# Should output: .../.venv/bin/python
```

Further 1D Probing

Upgraded Modular Workflow

Upgraded the pipeline to have a **modular observation function,** where I could use a **class based inheritance** to define different test functions in a highly modular fashion:

```
class BenchmarkFunctions:
    """
    A library of 1D benchmark functions for Bayesian Optimization testing.
    All functions accept a single float 'x' and return a float 'y'.
    """

    @staticmethod
    def double_peak(x):
        """
        Your custom test case:
        Peak 1 (Decoy): Broad hill at x=3.0, Height=10
        Peak 2 (True Max): Narrow spike at x=7.5, Height=12
        Domain: [0, 10]
        """
        peak1 = 10.0 * math.exp(-(x - 3.0)**2 / 2.0)
        peak2 = 12.0 * math.exp(-(x - 7.5)**2 / 0.5)
        return peak1 + peak2
    @staticmethod
    def rastrigin_1d(x):
        """
        The 'Egg Carton' - many local maxima.
        Domain: [-5.12, 5.12]
        Global Max (Inverted): x=0
        """
        # Inverted for maximization
        return -(10 + x**2 - 10 * np.cos(2 * np.pi * x)) + 20
    @staticmethod
    def schwefel_1d(x):
        """
        The 'Deceptive Trap' - max is at edge, deep local optima.
        Domain: [-500, 500]
        Global Max: x=420.96
        """
        return x * np.sin(np.sqrt(np.abs(x)))



CURRENT_TEST_FUNC = BenchmarkFunctions.double_peak



# ---------------------- Modular 1D Simulation Functions --------------------- #
def run_modular_simulation(parameters, target_func=CURRENT_TEST_FUNC,
noise_amplitude=NOISE_AMP, noiseless=False):
    """
    A generic wrapper that runs ANY 1D benchmark function passed to it.

    Args:
        parameters (dict): The dictionary from Ax (e.g., {'thickness': 3.5})
        target_func (callable): The function to run (e.g.,
BenchmarkFunctions.double_peak)
        noise_amplitude (float): The total range of noise.
        noiseless (bool): If True, returns pure data (good for plotting 'Truth').

    Returns:
        dict: The metric dictionary expected by Ax/BoTorch.
    """
    # 1. Explicitly extract 'thickness'
    # This ensures we are optimizing the right variable, regardless of dictionary order.
    x = parameters.get("thickness")

    if x is None:
        raise ValueError("The parameter 'thickness' was not found in the parameters
dictionary.")
    # 2. Run the "Imported" Physics Function
    pure_result = target_func(x)
```

```
    # 3. Handle Noise
    if noiseless:
        # Return pure result (useful for plotting the 'True Function' line)
        return {"tbr": pure_result, "noise_amplitude": noise_amplitude}

    # Gaussian Noise (3-Sigma Limit calculation)
    sigma = noise_amplitude / 3.0

    # Generate noise (using torch to match your original snippet)
    noise = torch.randn(1).item() * sigma
    # Return formatted result for Ax
    # Note: Ax usually likes (mean, sem) tuples, but your code uses simple dicts
    return {"tbr": pure_result + noise}
```

- Note that this function looks for a parameter called "thickness" defined with ax, however our only parameter defined is thickness

```
parameters=[
    {
        "name": "thickness",
        "type": "range",
        "bounds": DOMAIN,  # Search between 0cm and 10cm
        "value_type": "float",
    },
```

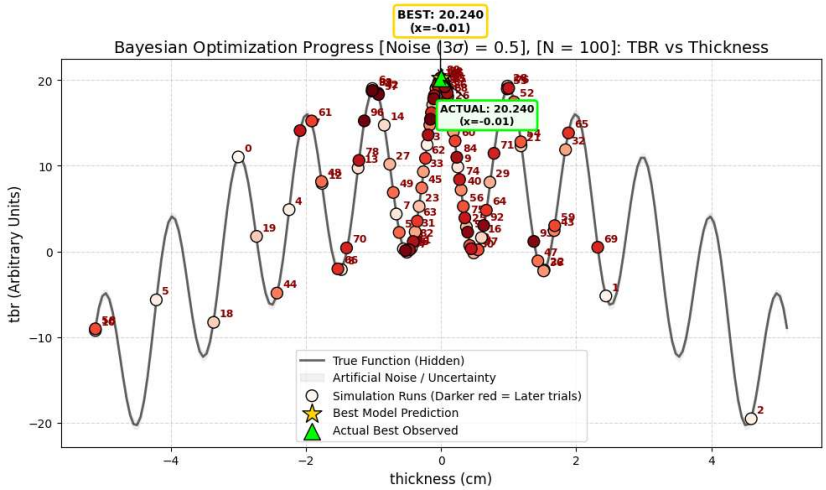
Harder Tests: The Modified Rastrigin (The "Egg Carton")

- **Why it's hard:** It has a cosine wave superimposed on a parabola. This creates many regularly spaced local maxima. It tests if your Acquisition Function is aggressive enough to jump out of a local "good enough" peak to find the "best" peak.
- **Domain:** [-5.12, 5.12]
- True optimum : x = 0.0, y = 20.0

N=100

For N (num trials)=100, it was not able to create a good GPR, it thought the sinusoid was noise

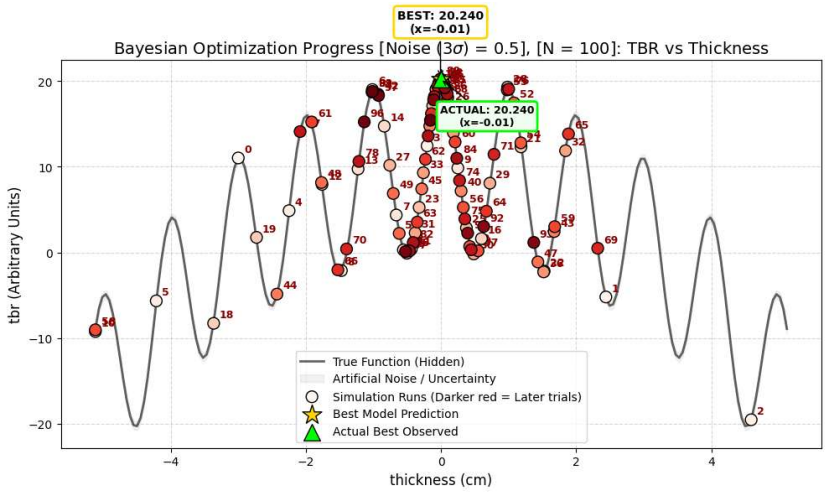
[WARNING 02-17 12:51:26] ax.adapter.cross_validation: Metric tbr was unable to be reliably fit.
[WARNING 02-17 12:51:26] ax.service.utils.best_point: Model fit is poor; falling back on raw data for best point.
[WARNING 02-17 12:51:26] ax.service.utils.best_point: Model fit is poor and data on objective metric tbr is noisy; interpret best points results carefully.



N=50

Same story

[WARNING 02-17 12:51:26] ax.adapter.cross_validation: Metric tbr was unable to be reliably fit.
[WARNING 02-17 12:51:26] ax.service.utils.best_point: Model fit is poor; falling back on raw data for best point.
[WARNING 02-17 12:51:26] ax.service.utils.best_point: Model fit is poor and data on objective metric tbr is noisy; interpret best points results carefully.



From an ML perspective this is a **fail** as it was not able to find a good surrogate model

- This why the "actual best" is the same as the "best model prediction". The code could not find a model optimum and so defaulted to an actual best. **This should not happen.**
- To improve this, hyperparameters would have to be tweaked to make the model more exploratory as opposed to optimising

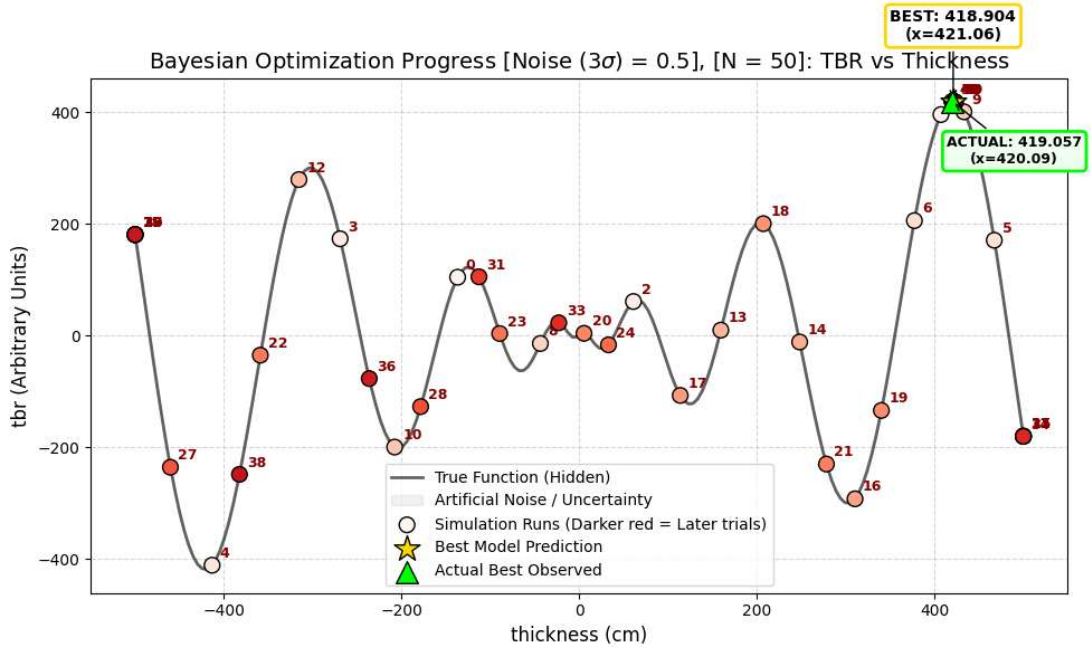
However from a modelling perspective it was able to efficiently able to find the global maximum and so is a **success**

So a **partial success.**

Harder Tests: The Schwefel Function (The "Deceptive" Trap)

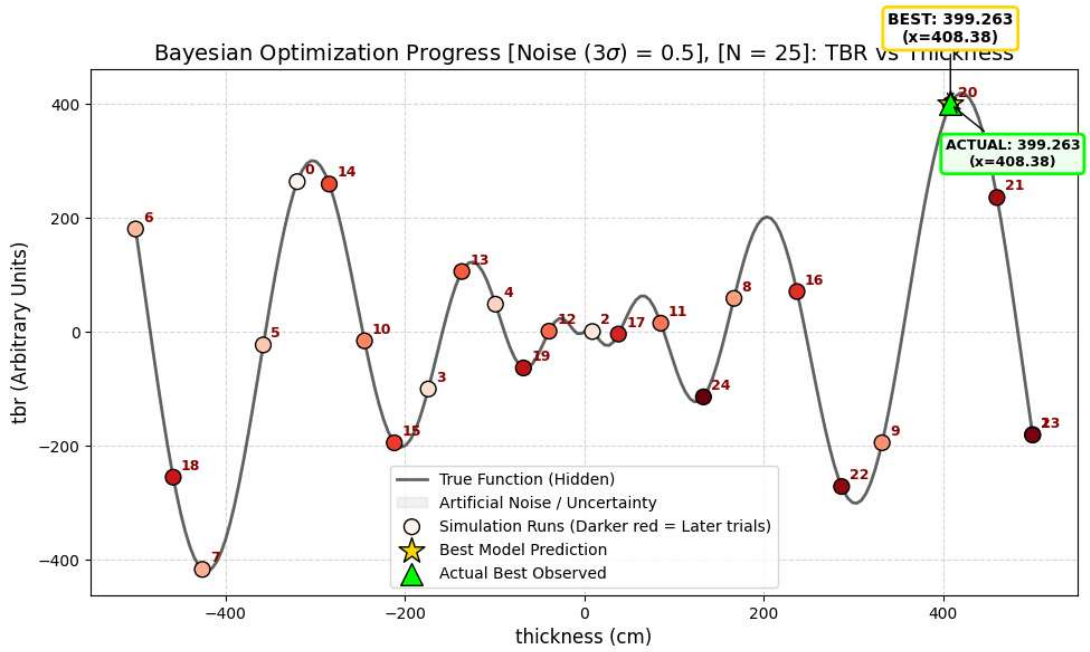
- **Why it's hard:** The global maximum is very close to the domain boundary, and the second-best maximum is far away. Algorithms often get stuck in the local optima because the gradient doesn't clearly point toward the global max until you are right on top of it.
- **Domain:** [-500, 500]
- True Optimal Thickness: 420.9687, giving 418.9829

N=50 (3sigma = 0.5)



- **Success!**

N=25 (3sigma = 0.5)

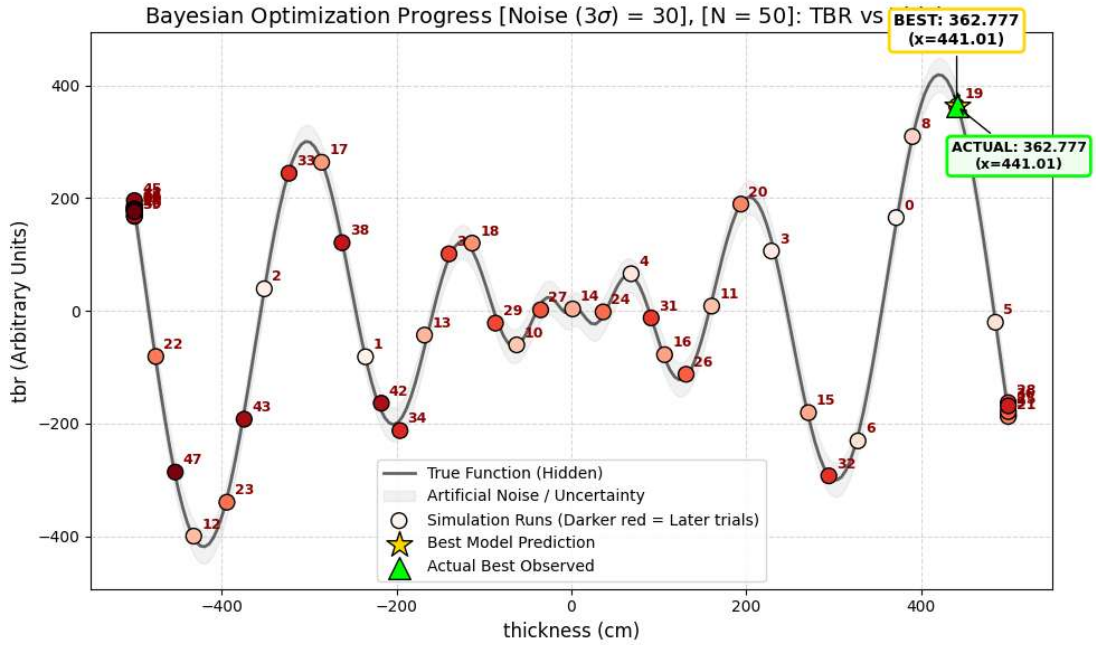


- **Same issue as before ("Model fitting fail")**

```
[WARNING 02-17 16:07:43] ax.adapter.cross_validation: Metric tbr was unable to be
reliably fit.
[WARNING 02-17 16:07:43] ax.service.utils.best_point: Model fit is poor; falling back on
raw data for best point.
[WARNING 02-17 16:07:43] ax.service.utils.best_point: Model fit is poor and data on
objective metric tbr is noisy; interpret best points results carefully.
```

N=50 (3sigma = 30)

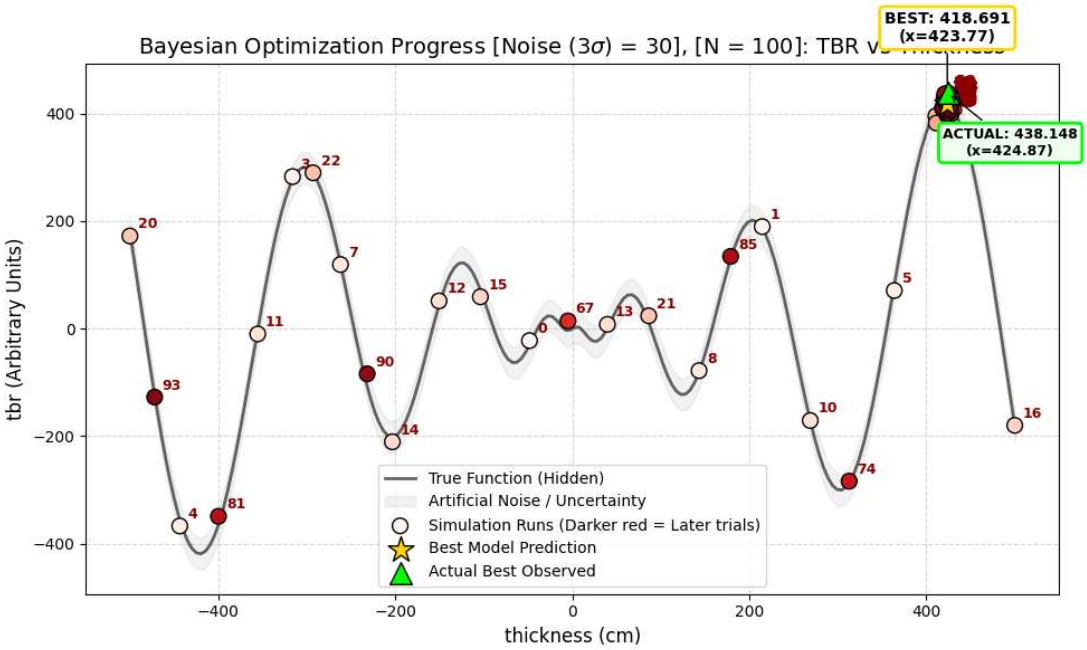
Bear in mind that the amplitude of this function is far greater.



- Model fitting fail.

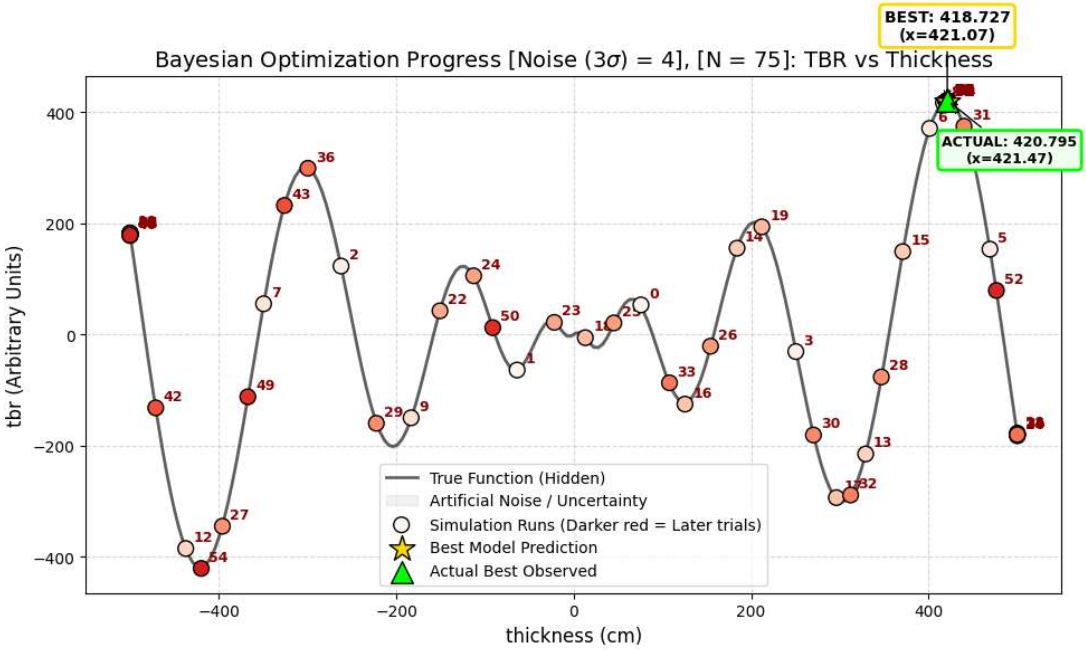
```
[WARNING 02-17 16:07:43] ax.adapter.cross_validation: Metric tbr was unable to be
reliably fit.
[WARNING 02-17 16:07:43] ax.service.utils.best_point: Model fit is poor; falling back on
raw data for best point.
[WARNING 02-17 16:07:43] ax.service.utils.best_point: Model fit is poor and data on
objective metric tbr is noisy; interpret best points results carefully.
```

N=100 (3sigma = 30)



- **Great success!**
- **Not only converged, but the model prediction was more accurate to the true maximum** (less noisy overshoot)
    - True Optimal Thickness: 420.9687, giving 418.9829
- For ~ 3% error, needed 100 trials

N=100 (3sigma = 4) (~ 0.5% error)

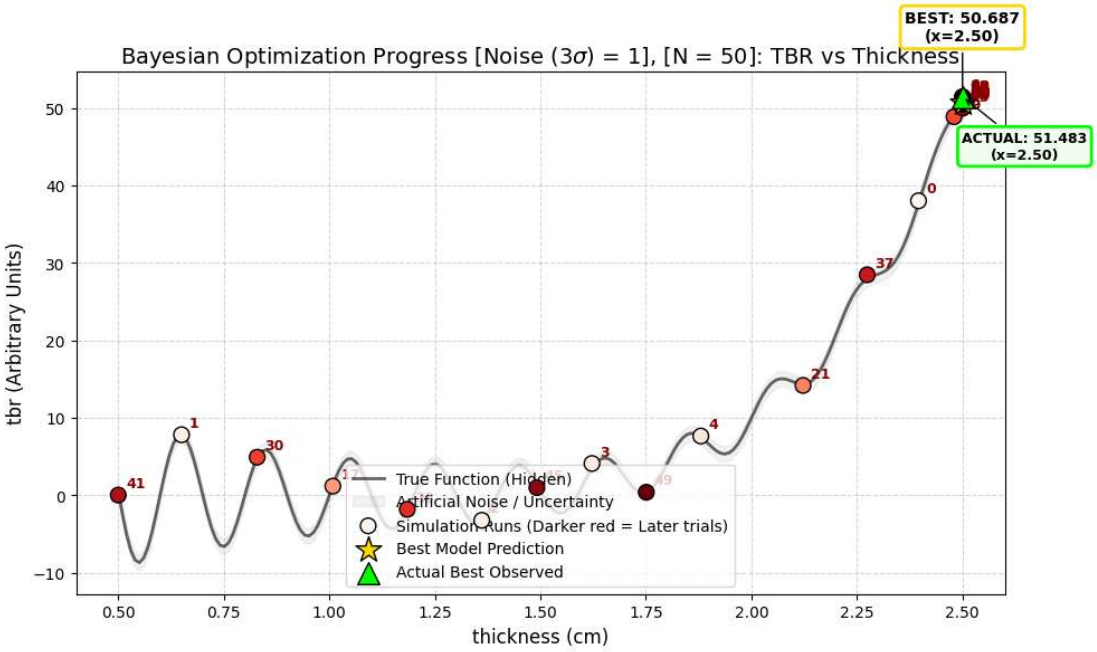


- **success!**
- As this noise is comparably to our model, this suggests we may need 25 additional iterations (if we start with 50 data points)

Harder Tests: The Gramacy & Lee Function (The "Variable Wiggle")

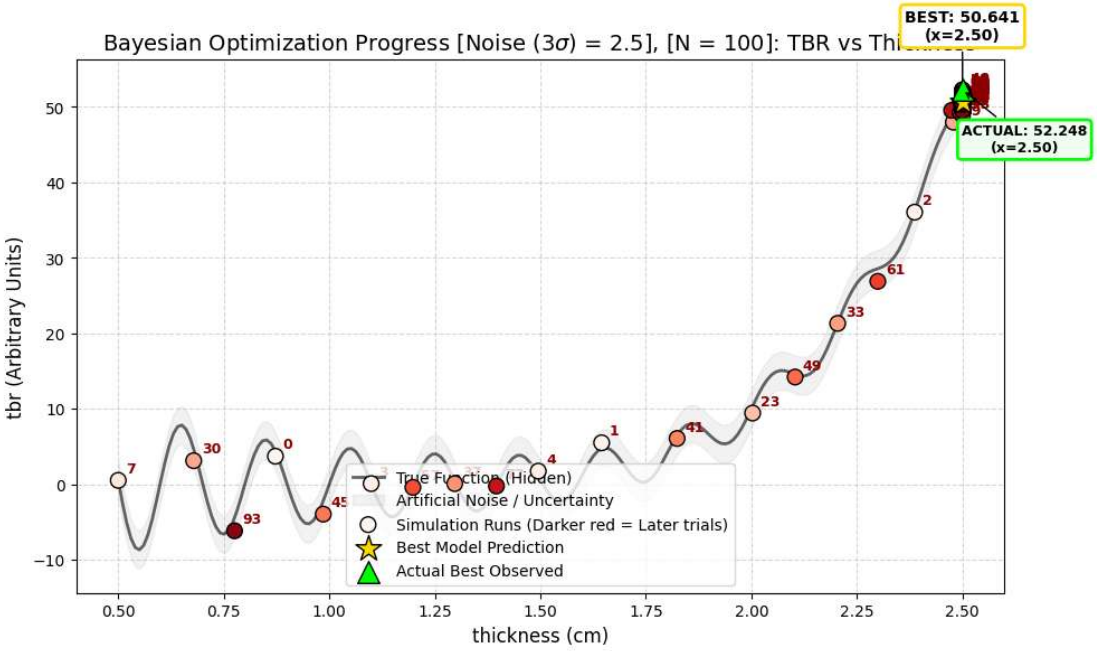
- **Why it's hard:** This is a classic 1D test for GPs. The function is smooth but changes "speed" (frequency) across the domain. Standard GPs with a constant lengthscale often fail here because they either over-smooth the wiggly part or over-fit the smooth part.
- **Domain:** [0.5, 2.5]
- True Optimum: x = 2.5, y ≈ 50.625 (at the boundary)

N=50, 3sigma = 1 (2% noise)



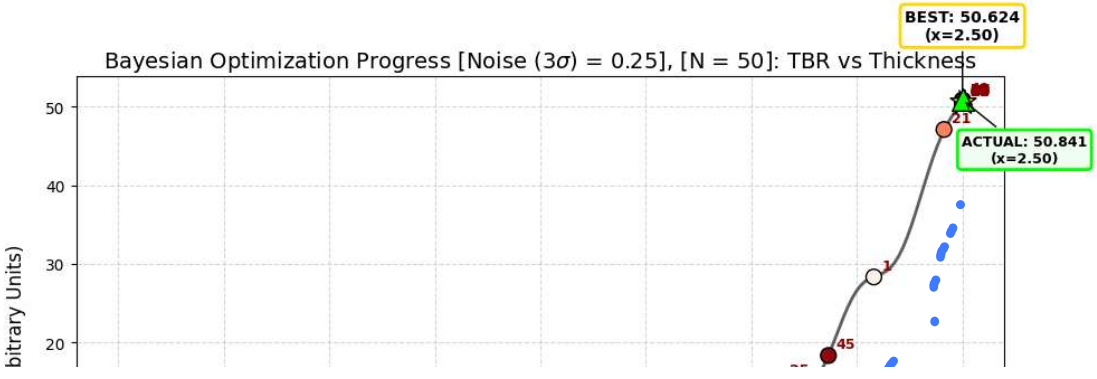
- **Great success!**
- **Not only converged, but the model prediction was more accurate to the true maximum** (less noisy overshoot)

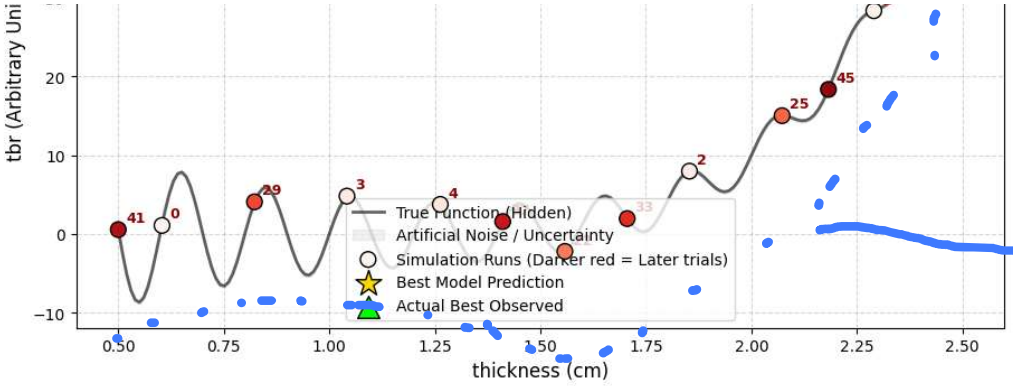
N=100, 3sigma = 2.5 (5% noise)



- **Great success!**
- **Not only converged, but the model prediction was more accurate to the true maximum** (less noisy overshoot)
- But needed 50 extra trials

N=100, 3sigma = 0.25 (0.5% noise)





Looks like a nyquist sampling error!

- **Great success!**
- **Not only converged, but the model prediction was more accurate to the true maximum** (less noisy overshoot)
- Was **extremely accurate**
  - True max: x = 2.5, y ≈ 50.625 (at the boundary)
- **Looks like it benefitted from nyquist sampling error**
  - **Will need to follow nyquist theory limit in future**